1. Introduction & Core Concepts

Object-Oriented Programming (OOP) is a paradigm organized around objects rather than actions, conceptually designed so that data controls access to code. This model aims to handle increasing system complexity by incorporating modularity, data protection, and reusability.

Classes

A class is a logical entity that serves as a blueprint or template for creating individual objects.

- **Structure:** A class bundles data members (variables) and methods (functions) together into a single unit.
- **Memory Allocation:** Defining a class merely creates a template; the class itself does not consume memory space.

Objects

An object is a real-world entity and a specific instance of a class.

- **Memory:** Unlike classes, an object takes up physical space in memory once created.
- **Characteristics:** Every object has three defining characteristics:
    - **State:** The data or value held by the object.
    - **Behavior:** The functionality or operations (methods) the object can perform.
    - **Identity:** A unique ID implemented internally by the Java Virtual Machine (JVM) to distinctly identify the object, even if its state is identical to another.

Constructors

Constructors are special member functions explicitly designed to initialize the objects of their class.

- **Rules:** A constructor must share the exact name of the class and is automatically invoked when an object is created. Modifiers such as abstract, static, final, or synchronized are not allowed on constructors.
- **Types:**

1. **Default (No-Arg) Constructor:** Takes no parameters. If you do not explicitly define a constructor, the Java compiler automatically provides a default one that initializes instance variables to default values (e.g., zero, null, or false).
2. **Parameterized Constructor:** Accepts specific parameters to provide distinct initial values to different objects upon creation.

- **Overloading:** A class can have multiple constructors with different parameter lists, allowing objects to be initialized in various ways.

Crucial Keywords

- **new:** This operator is used to dynamically allocate memory for an object at runtime and returns a memory reference to it. In Java, all class objects must be dynamically allocated using new.
- **this:** A reference keyword that points to the current object invoking the method. It is primarily used to resolve namespace collisions between instance variables and parameters, or to invoke the current class's constructor using this().
- **static:** A non-access modifier that dictates that a variable, method, block, or nested class belongs to the class itself rather than any specific object instance.
    - *Static Variables:* Only a single copy is created and shared across all instances of the class.
    - *Static Methods:* Can be invoked without creating an object of the class (e.g., classname.method()), but they can only directly access other static data and methods, and cannot use the this or super keywords

Kunal's notes:

- A **class** is a template for an object, and an **object** is an instance of a class.
- A class creates a new data type that can be used to create objects.
- When you declare an object of a class, you are creating an instance of that class.
- Thus, **a class is a logical construct**. An **object has physical reality**. (That is, an object occupies space in memory.)
- Objects are characterized by three essential **properties: state, identity, and behavior.**
- The **state of an object** is a value from its data type. The **identity of an object** distinguishes one object from another. It is useful to think of an object's identity as the

place where its value is stored in memory. The **behavior of an object** is the effect of data-type operations.

- The **dot operator** links the name of the object with the name of an instance variable.
- The variables stored inside each object are called **Instance variables.**
- Although commonly referred to as the dot operator, the formal specification for Java categorizes the . as a **separator**.
- **The 'New' keyword** used to create new objects. It dynamically allocates (that is, allocates at run time) memory for an object & returns a reference to it.
- This reference is, more or less, the address in memory of the object allocated by new. This reference is then stored in the variable.
- Thus, in Java, all class objects must be dynamically allocated.

Box mybox; // declare reference to object, mybox is reference of type Box.
mybox = new Box(); // allocate a Box object

- The first line declares mybox as a reference to an object of type Box. At this point, mybox does not yet refer to an actual object. The next line allocates an object and assigns a reference to it to mybox. After the second line executes, you can use mybox as if it were a Box object. But in reality, mybox simply holds, in essence, the memory address of the actual Box object.
- The key to Java's safety is that you cannot manipulate references as you can actual pointers. Thus, you cannot cause an object reference to point to an arbitrary memory location or manipulate it like an integer.

classname classvariable = new classname ( );

- Here, classvariable is a variable of the class type being created. The classname is the name of the class that is being instantiated. The class name followed by parentheses specifies the constructor for the class. A **constructor** defines what occurs when an object of a class is created.

- You might be wondering why you do not need to use new for such things as integers or characters. The answer is that Java's primitive types are not implemented as objects (Thus they are stored in stack memory only). Rather, they are implemented as "normal" variables. This is done in the interest of efficiency.
- It is important to understand that new allocates memory for an object during run time.

  Box b1 = new Box();
  Box b2 = b1;

- Here, b1 and b2 will both refer to the same object. The assignment of b1 to b2 did not allocate any memory or copy any part of the original object. It simply makes b2 refer to the same object as does b1. Thus, any changes made to the object through b2 will affect the object to which b1 is referring, since they are the same object.
- When you assign one object reference variable to another object reference variable, you are not creating a copy of the object, you are only making a copy of the reference.

```
int square(int i){
   return i * i;
}
```

- A **parameter** is a variable defined by a method that receives a value when the method is called. For example, in square(int i), i is a parameter. An argument is a value that is passed to a method when it is invoked. If square(100) passes, 100 as an argument. Inside square( ), the parameter i receives that value.
- NOTE:
  Bus bus = new Bus(); //LHS (reference i.e. bus) is looked by compiler & RHS (object i.e. new Bus()) is looked by JVM
- **The this Keyword**: Sometimes a method will need to refer to the object that invoked it. To allow this, Java defines the this keyword.
- This can be used inside any method to refer to the current object. That is, this is always a reference to the object on which the method was invoked.

- **The final Keyword**: A field can be declared as final. Doing so prevents its contents from being modified, making it, essentially, a constant.
- This means that you must initialize a final field when it is declared.
- It is a common coding convention to choose all uppercase identifiers for final fields:

```
final int INCREASE = 2;
```

- Unfortunately, final guarantees immutability only when instance variables are primitive types, not reference type of object.
- If an instance variable of a reference type has the final modifier, the value of that instance variable (the reference to an object) will never change—it will always refer to the same object—but the value of the object itself can change.
- **The finalize( ) Method:** Sometimes an object will need to perform some action when it is destroyed.
- To handle such situations, Java provides a mechanism called finalization. By using finalization, you can define specific actions that will occur when an object is just about to be reclaimed by the garbage collector.
- To add a finalizer to a class, you simply define the finalize( ) method. The Java run time calls that method whenever it is about to recycle an object of that class. Right before an asset is freed, the Java run time calls the finalize( ) method on the object.

```
protected void finalize( ) {
   // finalization code here
}
```

- **Constructors:** Once defined, the constructor is automatically called when the object is created, before the new operator completes.
- Constructors look a little strange because they have no return type, not even void.
- This is because the implicit return type of a class' constructor is the class type itself.
- In the line

Box mybox1 = new Box(); //new Box( ) is calling the Box( ) constructor.

- **Inheritance and constructors in Java:** In Java, constructor of base class(Parent class) with no argument gets automatically called in derived class constructor.
- For example, output of following program given below is:
  Base Class Constructor Called
  Derived Class Constructor Called

  // filename: Main.java

```java
class Base {
   Base() {
       System.out.println("Base Class Constructor Called ");
   }
}
class Derived extends Base {
   Derived() {
       System.out.println("Derived Class Constructor Called ");
   }
}
public class Main {
   public static void main(String[] args) {
       Derived d = new Derived();
   }
}
```

- Any class will have a default constructor, it does not matter if we declare it in the class or not. If we inherit a class, then the derived class must call its super class constructor. It is done by default in derived class.
- If it does not have a default constructor in the derived class, the JVM will invoke its default constructor and call the super class constructor by default. If we have a parameterised constructor in the derived class still it calls the default super class constructor by default.

- In this case, if the super class does not have a default constructor, instead it has a parameterised constructor, then the derived class constructor should explicitly call the parameterised super class constructor.

2. Packages, Static, Singleton, Inbuilt functions

1. Packages

A Java package is a namespace that groups similar types of classes, interfaces, and sub-packages together. They can be categorized into built-in packages (like java.lang, java.util, java.io) and user-defined packages.

- **Core Advantages:** Packages categorize classes to make them easily maintainable, provide access protection, and remove naming collisions (allowing classes with the same name to exist in different packages).
- **Implementation:** To define a package, the package command must be the absolute first statement in your Java source file. If this statement is omitted, the classes default to an unnamed default package.
- **Hierarchy:** Packages can be multi-leveled (e.g., package java.awt.image;), which must strictly reflect the physical directory structure on your file system.

2. Static Keyword

The static keyword designates that a member (variable, method, or nested class) belongs to the class itself, rather than to instances (objects) of that class.

- **Static Nested Classes:** A static nested class acts like a static member of the outer class and does not require an instance of the outer class to be instantiated.
- **Interface Variables:** By default, any variable declared inside an interface is implicitly public, static, and final. They must be initialized at the time of declaration.

3. Singleton Concept

While not explicitly named as a standalone unit in your coursework, the Singleton is a foundational Object-Oriented design pattern directly heavily reliant on the static keyword. It restricts a class to instantiating only *one* single object.

- **How it works:**
  1. Define a private constructor so no external class can use the new keyword to create an object.
  2. Create a private static instance variable of the class itself.
  3. Expose a public static method (often named getInstance()) that returns that single static instance, creating it only if it doesn't already exist.

4. Inbuilt Functions

Inbuilt functions (or pre-defined methods) are methods provided by Java's standard API packages.

- **String Methods:** Your coursework heavily features inbuilt functions from the String class, such as indexOf() to find character indices, lastIndexOf(), startsWith(), and endsWith() to check string literals.
- **Utility Methods:** Classes like ArrayList have built-in functions such as ensureCapacity() and trimToSize() to manage dynamic memory allocation automatically.

Kunal's notes:

- **Packages** are containers for classes. They are used to keep the class name space compartmentalized. (Basically arranged in folders)
- For example, a package allows you to create a class named List, which you can store in your own package without concern that it will collide with some other class named List stored elsewhere.
- **Packages** are stored in a hierarchical manner and are explicitly imported into new class definitions.
- The package is both a naming and a visibility control mechanism.
- The following statement creates a package called MyPackage: **package MyPackage;**
- Java uses file system directories to store packages. For example, the .class files for any classes you declare to be part of MyPackage must be stored in a directory called MyPackage.
- Remember that case is significant, and the directory name must match the package name exactly.
- A package hierarchy must be reflected in the file system of your Java development system.
- For example, a package declared as **package java.awt.image;** needs to be stored in java\awt\image in a Windows environment. Be sure to choose your package names carefully.
- You cannot rename a package without renaming the directory in which the classes are stored.

How does the Java run-time system know where to look for packages that you create? The answer has three parts.

- First, by default, the Java run-time system uses the current working directory as its starting point.

Thus, if your package is in a subdirectory of the current directory, it will be found.

- Second, you can specify a directory path or paths by setting the CLASSPATH environmental variable.

- Third, you can use the -classpath option with java and javac to specify the path to your classes.

- When a package is imported, only those items within the package declared as public will be available to non-subclasses in the importing code.

Understanding static:

- When a member is declared **static**, it can be accessed before any objects of its class are created, and without reference to any object. You can declare both methods and variables to be static. (can access and modify the static variables)
- The most common example of a static member is main( ). main( ) is declared as static because it must be called before any objects exist.
- **Static method** in Java is a method which belongs to the class and not to the object.
- A **static method** can access only static data. It cannot access non-static data (it belongs to instance/objects)
- A **non-static member** belongs to an instance(In Java, an instance is an individual object created from a class). It's meaningless without somehow resolving which instance of a class you are talking about.
- In a static context, you don't have an instance, that's why you can't access a non-static member without explicitly mentioning an object reference.
- In fact, you can access a non-static member in a static context by specifying the object reference explicitly:

```
public static void main(String[] args) {
   Workout displayobj = new Workout();
```

```
    displayobj.display("Coding");
}
void display(String work){
    System.out.println(work);
}
```

- A static method can call only other static methods and cannot call a non-static method from it.
- A static method can be accessed directly by the class name and doesn't need any object and they cannot be referred to by "this" or "super" keywords.
- Because 'this' basically represents an object and 'static' does not depend on object.
- If you need to do computation in order to initialize your static variables, you can declare a static block that gets executed exactly once, when the class is first loaded.
- Only **Inner class** can be static. Outside the class, static cannot be used.
- **Inner classes** can be dependent on outside methods, But outside classes cannot be dependent on inner classes/methods.

```
// Demonstrate static variables, methods, and blocks.
  class StaticBlock{
    static int a = 3;
    static int b;
    static void meth(int x) {
      System.out.println("x = " + x);
      System.out.println("a = " + a);
      System.out.println("b = " + b);
    }
    static {
      System.out.println("Static block initialized.");
      b = a * 4;
    }
    public static void main(String args[]) {
      meth(42);
```

```
    }
}
```

- As soon as the UseStatic class is loaded, all of the static statements are run. First, a is set to 3, then the static block executes, which prints a message and then initializes b to a*4 or 12. Then main( ) is called, which calls meth( ), passing 42 to x. The three println( ) statements refer to the two static variables a and b, as well as to the local variable x.
- Here is the output of the program:

  Static block initialized. x = 42

  a = 3

  b = 12

Note: main method is static, since it must be accessible for an application to run, before any instantiation takes place.

NOTE: Only nested classes can be static.

NOTE: Static inner classes can have static variables.

- You can't override the inherited static methods, as in java overriding takes place by resolving the type of object at run-time and not compile time, and then calling the respective method.
- Static methods are class level methods, so it is always resolved during compile time.
- Static INTERFACE METHODS are not inherited by either an implementing class or a sub-interface.

```
public class Static {
   // class Test  // ERROR
   static class Test{
      String name;
      public Test(String name) {
         this.name = name;
      }
   }
```

```
    public static void main(String[] args) {
        Test a = new Test("Kunal");
        Test b = new Test("Rahul");
        System.out.println(a.name); // Kunal
        System.out.println(b.name); // Rahul
    }
}
```

- Because the static keyword may modify the declaration of a member type within the body of a non-inner class or interface. (In this case, name is first null and then it becomes the pointed name of the variables)
- Here, test does not have any instance of its outer class Static. Neither does main. But main & Test can have instances of each other.
- **Singleton Class** are the class which can create only one object of the class.

3. Principles of OOP

1. Inheritance (The "IS-A" Relationship)

Inheritance is the mechanism where one class (the child or subclass) acquires all the properties and behaviors of another class (the parent or superclass). This represents an "IS-A" relationship (e.g., a ComputeEngine IS-A CloudResource).

- **Purpose:** It promotes code reusability and establishes a natural hierarchy, which is crucial for achieving runtime polymorphism.
- **Types in Java:** Java natively supports Single, Multilevel, and Hierarchical inheritance through classes.
- **The Multiple Inheritance Problem:** To prevent the "Diamond Problem" (ambiguity when multiple parent classes have the same method), Java does not support Multiple or Hybrid inheritance directly through classes. Instead, these are achieved strictly through Interfaces.

2. Polymorphism (One Interface, Multiple Forms)

Polymorphism allows objects of different classes to be treated as objects of a common superclass. It dictates how a single action can be performed in different ways.

- **Compile-Time (Static) Polymorphism:** Achieved via Method Overloading. This occurs when two or more methods within the same class have the exact same name but different

parameter declarations (changing the number or type of arguments). The compiler determines which method to execute based on the method signature.

- **Run-Time (Dynamic) Polymorphism:** Achieved via Method Overriding and Dynamic Method Dispatch. Overriding happens when a subclass provides a specific implementation of a method already defined by its superclass (same name and type signature). During execution, if an overridden method is called through a superclass reference variable referring to a subclass object (Upcasting), Java dynamically determines which version of the method to execute based on the *actual object type* at runtime, not the reference type.

3. Abstraction (Hiding Complexity)

Abstraction is the process of hiding the internal implementation details and exposing only the essential functionality to the user. It lets the developer focus on *what* the object does rather than *how* it does it.

- **Abstract Classes:** Declared with the abstract keyword, these classes cannot be instantiated directly and can contain both abstract methods (methods without a body) and non-abstract (concrete) methods. They provide partial abstraction.
- **Interfaces:** Act as a strict blueprint of a class. Prior to Java 8, interfaces provided 100% abstraction, meaning all methods were strictly abstract and public by default, and all variables were public, static, and final. A class using an interface must implement all of its methods.

4. Encapsulation (Data Hiding)

Encapsulation is the technique of wrapping the data (variables) and the code acting on the data (methods) together into a single, cohesive unit.

- **Implementation:** It is achieved by declaring class variables as private (restricting direct external access) and providing public getter and setter methods to modify and view the variable values safely.
- **Benefits:** This creates a protective shield that prevents the data from being accessed or **modified** by code outside the shield, ensuring data integrity and allowing the class to enforce validation rules.

Kunal's notes:

- **Inheritance:** To inherit a class, you simply incorporate the definition of one class into another by using the extends keyword.

  class "subclass-name" extends "superclass-name" { // body of class

  }

- You can only specify one superclass for any subclass that you create. Java does not support the inheritance of multiple superclasses into a single subclass.
- You can, as stated, create a hierarchy of inheritance in which a subclass becomes a superclass of another subclass.
- However, no class can be a superclass of itself.
- Although a subclass includes all of the members of its superclass, it cannot access those members of the superclass that have been declared as private.
- A Superclass Variable Can Reference a Subclass Object(Later you will understand that it is polymorphism): It is important to understand that it is the type of the reference variable—not the type of the object that it refers to—that determines what members can be accessed.
- When a reference to a subclass object is assigned to a superclass reference variable, you will have access only to those parts of the object defined by the superclass.

plainbox = weightbox;

(superclass) (subclass)

SUPERCLASS ref = new SUBCLASS(); // HERE ref can only access methods which are available in SUPERCLASS

- **super keyword**: Whenever a subclass needs to refer to its immediate superclass, it can do so by use of the keyword super.
- super has two general forms. The first calls the superclass' constructor.
- The second is used to access a member of the superclass that has been hidden by a member of a subclass. Sometimes a subclass defines a variable or method with the same name as the superclass. super lets you access the parent version.
- **Example**:

  BoxWeight(double w, double h, double d, double m) {

```
    super(w, h, d); // call superclass constructor
    weight = m;
  }
```

- Here, BoxWeight( ) calls super( ) with the arguments w, h, and d. This causes the Box constructor to be called, which initializes width, height, and depth using these values.
- BoxWeight no longer initializes these values itself. It only needs to initialize the value unique to it: weight. This leaves Box free to make these values private if desired.
- Thus, super( ) always refers to the superclass immediately above the calling class.
- This is true even in a multileveled hierarchy.

```
class Box {
    private double width;
    private double height;
    private double depth;
    // construct clone of an object
    Box(Box ob) { // pass object to constructor
      width = ob.width;
      height = ob.height;
      depth = ob.depth;
    }
}
class BoxWeight extends Box {
    double weight; // weight of box
    // construct clone of an object
    BoxWeight(BoxWeight ob) { // pass object to constructor
      super(ob);
```

```
        weight = ob.weight;
    }
}
```

- Notice that super() is passed an object of type BoxWeight—not of type Box. This still invokes the constructor Box(Box ob).
- **NOTE:** A superclass variable can be used to reference any object derived from that superclass.
- Thus, we are able to pass a BoxWeight object to the Box constructor. Of course, Box only has knowledge of its own members.
- **A Second Use for super:** The second form of super acts somewhat like this, except that it always refers to the superclass of the subclass in which it is used.

  super.member

- Here, member can be either a method or an instance variable. This second form of super is most applicable to situations in which member names of a subclass hide members by the same name in the superclass.
- super( ) always refers to the constructor in the closest superclass. The super( ) in BoxPrice calls the constructor in BoxWeight. The super( ) in BoxWeight calls the constructor in Box.
- In a class hierarchy, if a superclass constructor requires parameters, then all subclasses must pass those parameters "up the line." This is true whether or not a subclass needs parameters of its own.
- If you think about it, it makes sense that constructors complete their execution in order of derivation. Because a superclass has no knowledge of any subclass, any initialization it needs to perform is separate from and possibly prerequisite to any initialization performed by the subclass. Therefore, it must complete its execution first.
- **NOTE:** If super() is not used in the subclass' constructor, then the default or parameterless constructor of each superclass will be executed.
- **Using final with Inheritance:** The keyword final has three uses: **#1**, it can be used to create the equivalent of a named constant.

- **#2 Using final to prevent overriding:** To disallow a method from being overridden, specify final as a modifier at the start of its declaration. Methods declared as final cannot be overridden.
- Methods declared as final can sometimes provide a performance enhancement: The compiler is free to inline calls to them because it "knows" they will not be overridden by a subclass.
- When a small final method is called, often the Java compiler can copy the bytecode for the subroutine directly inline with the compiled code of the calling method, thus eliminating the costly overhead associated with a method call. Inlining is an option only with final methods.
- Normally, Java resolves calls to methods dynamically, at run time. This is called **late binding.**
- However, since final methods cannot be overridden, a call to one can be resolved at compile time. This is called **early binding.**
- **#3 Using final to prevent inheritance:** Sometimes you will want to prevent a class from being inherited. To do this, precede the class declaration with final.
- **NOTE:** Declaring a class as final implicitly declares all of its methods as final, too.
- As you might expect, it is illegal to declare a class as both abstract and final since an abstract class is incomplete by itself & relies upon its subclasses to provide complete implementations.
- **NOTE:** Although static methods can be inherited, there is no point in overriding them in child classes because the method in parent class will always run no matter from which object you call it. That is why static interface methods cannot be inherited because these method will run from the parent interface and no matter if we were allowed to override them, they will always run the method in parent interface. That is why static interface method must have a body.
- **NOTE:** Polymorphism does not apply to instance variables.
- **Overloading:** it is possible to define two or more methods within the same class that share the same name, as long as their parameter declarations are different.
- While overloaded methods may have different return types, the return type alone is insufficient to distinguish two versions of a method.

- When Java encounters a call to an overloaded method, it simply executes the version of the method whose parameters match the arguments used in the call.
- In some cases, Java's automatic type conversions can play a role in overload resolution.

```
class OverloadDemo {
   void test(double a){
      System.out.println("Inside test(double) a: " + a);
   }
}
class Overload {
    public static void main(String args[]) {
       OverloadDemo ob = new OverloadDemo();
       int i = 88;
       ob.test(i);        // this will invoke test(double)
       ob.test(123.2);    // this will invoke test(double)
    }
}
```

- As you can see, this version of OverloadDemo does not define test(int). Therefore, when test( ) is called with an integer argument inside Overload, no matching method is found.
- However, Java can automatically convert an integer into a double, and this conversion can be used to resolve the call. Therefore, after test(int) is not found, Java elevates i to double and then calls test(double).
- Of course, if test(int) had been defined, it would have been called instead. Java will employ its automatic type conversions only if no exact match is found.
- **Returning Objects:**

```
// Returning an object.
    class Test {
       int a;
         Test(int i) {
```

```
        a = i;
      }
      Test incrByTen() {
        Test temp = new Test(a+10);
        return temp;
      }
    }
    class RetOb {
      public static void main(String args[]) {
        Test ob1 = new Test(2);
        Test ob2;
        ob2 = ob1.incrByTen();
        System.out.println("ob1.a: " + ob1.a);
        System.out.println("ob2.a: " + ob2.a);
      }
}
```

- **Output:**

  ob1.a: 2

  ob2.a: 12

- As you can see, each time incrByTen( ) is invoked, a new object is created, and a reference to it is returned to the calling routine. Since all objects are dynamically allocated using new, you don't need to worry about an object going out-of-scope because the method in which it was created terminates. The object will continue to exist as long as there is a reference to it somewhere in your program. When there are no references to it, the object will be reclaimed the next time garbage collection takes place.

- **Overriding:** In a class hierarchy, when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to override the method in the superclass.
- When an overridden method is called from within its subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden.
- Method overriding occurs only when the names and the type signatures of the two methods are identical. If they are not, then the two methods are simply overloaded.

(Check display functions in box classes)

- **Dynamic Method Dispatch:** Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.
- Dynamic method dispatch is important because this is how Java implements run-time polymorphism.
- **Let's begin by restating an important principle:** a superclass reference variable can refer to a subclass object.
- When an overridden method is called through a superclass reference, Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs. Thus, this determination is made at run time.
- In other words, it is the type of the object being referred to (not the type of the reference variable) that determines which version of an overridden method will be executed.
- If B extends A then you can override a method in A through B with changing the return type of method to B.

Abstraction	Encapsulation
Abstraction is a feature of OOPs that hides the **unnecessary** detail but shows the essential information.	Encapsulation is also a feature of OOPs. It hides the code and data into a **single** entity or unit so that the data can be protected from the outside world.
It solves an issue at the **design** level.	Encapsulation solves an issue at **implementation** level.
It focuses on the **external** lookout.	It focuses on **internal** working.
It can be implemented using **abstract classes** and **interfaces**.	It can be implemented by using the **access modifiers** (private, public, protected).
It is the process of **gaining** information.	It is the process of **containing** the information.
In abstraction, we use **abstract classes** and **interfaces** to hide the code complexities.	We use the **getters** and **setters** methods to hide the data.
The objects are **encapsulated** that helps to perform abstraction.	The object need not to **abstract** that result in encapsulation.

4. Access control, Inbuilt packages, Object class.

1. Access Control (Modifiers)

Access control determines the visibility and accessibility of classes, methods, constructors, and variables. It is the primary mechanism for implementing **Encapsulation** (data hiding). Java provides four levels of access control:

private: The most restrictive level. The member is accessible *only* within the exact class where it is declared. It cannot be accessed by subclasses or other classes in the same package.

default (Package-Private): If no modifier is explicitly specified, the member takes on default access. It is accessible only to classes within the *same package*.

protected: The member is accessible within its own package (like default) *and* by any subclass of its class, even if that subclass resides in a completely different package.

public: The least restrictive level. The class, method, or variable is globally accessible from any other class in any package across the application.

2. Inbuilt Packages

A package in Java is a namespace used to group related classes and interfaces, providing access protection and preventing naming collisions.

Java comes with a massive, pre-written standard library known as **Inbuilt Packages** (or the Java API). Some of the most critical built-in packages include:

java.lang: The core package that contains fundamental classes like String, Math, System, and Thread. It is automatically imported into every Java program by the compiler.

java.util: Contains collection frameworks (ArrayList, HashMap), legacy collection classes, event model, date and time facilities, and utility classes (like Scanner).

java.io: Provides system input and output through data streams, serialization, and the file system.

java.awt & javax.swing: Used for building Graphical User Interfaces (GUIs).

3. The Object Class

The java.lang.Object class is the cosmic superclass in Java. **Every single class in Java implicitly extends the Object class**, making it the absolute root of the Java class hierarchy. If you define a class without an extends keyword, the compiler automatically adds extends Object behind the scenes.

Because every class inherits from Object, every object in Java comes with a set of inherited built-in methods designed for general utility. The most commonly overridden ones are:

toString(): Returns a string representation of the object. By default, it returns the class name followed by the "@" character and the unsigned hexadecimal representation of the hash code. It is highly recommended to override this to return meaningful data.

equals(Object obj): Indicates whether some other object is "equal to" this one. By default, it checks for memory reference equality (if two variables point to the exact same object in memory). It is overridden to check for logical equality (e.g., two different employee objects having the same ID).

hashCode(): Returns an integer hash code value for the object, which is strictly required if the object will be used in hash-based collections like HashMap or HashSet.

clone(): Creates and returns a copy of the object.

Kunal's notes:

- **Access Control:** How a member can be accessed is determined by the access modifier attached to its declaration. Usually, you will want to restrict access to the data members of a class—allowing access only through methods. Also, there will be times when you will want to define methods that are private to a class.
- Java's access modifiers are public, private, and protected. Java also defines a default access level. protected applies only when inheritance is involved.
- When no access modifier is used, then by default the member of a class is public within its own package, but cannot be accessed outside of its package.

	Class	Package	Subclass (same pkg)	Subclass (diff pkg)	World (diff pkg & not subclass)
public	+	+	+	+	+
protected	+	+	+	+	
no modifier	+	+	+		
private	+				

+ : accessible
blank : not accessible

Most Restrictive ⟷ Least Restrictive

Access Modifiers ->	private	Default/no-access	protected	public
Inside class	Y	Y	Y	Y
Same Package Class	N	Y	Y	Y
Same Package Sub-Class	N	Y	Y	Y
Other Package Class	N	N	N	Y
Other Package Sub-Class	N	N	Y	Y

Same rules apply for inner classes too, they are also treated as outer class properties

Code:

```
package packageOne;
public class Base
{
   protected void display(){
      System.out.println("in Base");
   }
}
package packageTwo;
public class Derived extends packageOne.Base{
   public void show(){
      new Base().display();      // this is not working
      new Derived().display();   // is working
      display();//is working
   }
```

}

- protected allows access from subclasses and from other classes in the same package.
- We can use child class to use protected outside the package but only child class object can access it.
- That's why any Derived class instance can access the protected method in Base.
- The other line creates a Base instance (not a Derived instance!!). And access to protected methods of that instance is only allowed from objects of the same package.

display();

- allowed, because the caller, an instance of Derived has access to protected members and fields of its subclasses, even if they're in different packages

new Derived().display();

- allowed, because you call the method on an instance of Derived and that instance has access to the protected methods of its subclasses

new Base().display();

- not allowed because the caller's (the this instance) class is not defined in the same package like the Base class, so this can't access the protected method. And it doesn't matter - as we see - that the current subclasses a class from that package. That backdoor is closed ;)
- Remember that any time you talk about a subclass having access to a superclass member, we could be talking about the subclass inheriting the member, not simply accessing the member through a reference to an instance of the superclass.

class C

protected member;

// in a different package

class S extends C

obj.member; // only allowed if type of obj is S or subclass of S

- The motivation is probably as follows. If obj is an S, class S has sufficient knowledge of its internals, it has the right to manipulate its members, and it can do this safely.
- If obj is not an S, it's probably another subclass S2 of C, which S has no idea of.
- S2 may have not even been born when S is written. For S to manipulate S2's protected internals is quite dangerous. If this is allowed, from S2's point of view, it doesn't know who will tamper with its protected internals and how, this makes S2's job very hard to reason about its own state.
- Now if obj is D, and D extends S, is it dangerous for S to access obj.member? Not really. How S uses member is a shared knowledge of S and all its subclasses, including D. S as the superclass has the right to define behaviours, and D as the subclass has the obligation to accept and conform.
- For easier understanding, the rule should really be simplified to require obj's (static) type to be exactly S. After all, it's very unusual and inappropriate for subclass D to appear in S. And even if it happens, that the static type of obj is D, our simplified rule can deal with it easily by upcasting: ((S)obj).member.

5. Abstract class, Interface, Annotations

1. Abstract Class (Partial Abstraction)

An abstract class is a restricted class that cannot be directly instantiated (you cannot use the new keyword to create an object of it). It serves as a foundational template for other classes to extend.

- **Declaration:** It is declared using the abstract keyword.
- **Method Composition:** It can contain a mix of **abstract methods** (methods declared without a body, forcing subclasses to provide the implementation) and **concrete methods** (regular methods with a body).
- **State and Initialization:** Unlike interfaces, an abstract class can possess instance variables, data members, and its own constructors. When a subclass is instantiated, the abstract superclass's constructor is invoked.
- **Usage Context:** Use an abstract class when subclasses share a lot of common code or state, but still need to implement specific behaviors uniquely.

2. Interface (Total Abstraction & Multiple Inheritance)

An interface is a strict blueprint of a class used to achieve total abstraction and resolve Java's restriction on multiple class inheritance.

- **Declaration & Implementation:** Declared using the interface keyword. Classes consume interfaces using the implements keyword, while an interface can inherit from another interface using the extends keyword.
- **Implicit Modifiers:** By default, all fields (variables) inside an interface are implicitly public, static, and final (they are constants and must be initialized). All methods are implicitly public and abstract (prior to modern Java updates introducing default methods).
- **Multiple Inheritance:** While a Java class can only extend one parent class, it can implement an unlimited number of interfaces. This is how Java safely handles multiple inheritance without falling into the "Diamond Problem" of ambiguous method calls. If two implemented interfaces contain the exact same method signature, providing one implementation in the class satisfies both.

3. Annotations (Code Metadata)

Annotations provide data about a program that is not part of the program itself. They have no direct effect on the operation of the code they annotate but provide crucial instructions to the compiler or runtime environments.

- **The @Override Annotation:** In the context of inheritance and abstract implementations, you use the @Override annotation above a method to explicitly declare that this method is overriding a method in its superclass or interface.
- **Technical Benefit:** If you mark a method with @Override but make a typo in the method signature, the compiler will immediately throw an error, preventing subtle runtime bugs where the program accidentally creates a new overloaded method instead of overriding the intended one.

Kunal's notes:

- Sometimes you will want to create a superclass that only defines a generalized form that will be shared by all of its subclasses, leaving it to each subclass to fill in the details. Such a class determines the nature of the methods that the subclasses must implement.
- You may have methods that must be overridden by the subclass in order for the subclass to have any meaning. In this case, you want some way to ensure that a subclass does, indeed, override all necessary methods.
- Java's solution to this problem is the **abstract method**.
- You can require that certain methods be overridden by subclasses by specifying the abstract type modifier.

abstract Returntype Variablename(parameter-list);

- These methods are sometimes referred to as subclass's responsibility because they have no implementation specified in the superclass.
- Thus, a subclass must override them—it cannot simply use the version defined in the superclass.
- Any class that contains one or more abstract methods must also be declared abstract.

There can be no objects of an abstract class.

You cannot declare abstract constructors, or abstract static methods.

You can declare static methods in abstract class.

- Because there can be no objects for abstract class. If they had allowed us to call abstract static methods, it would mean we are calling an empty method (abstract) through classname because it is static.
- Any subclass of an abstract class must either implement all of the abstract methods in the superclass, or be declared abstract itself.
- Abstract classes can include as much implementation as they see fit i.e.there can be concrete methods(methods with body) in abstract class.
- Although abstract classes cannot be used to instantiate objects, they can be used to create object references, because Java's approach to run-time polymorphism is implemented through the use of superclass references.
- A public constructor on an abstract class doesn't make any sense because you can't instantiate an abstract class directly (can only instantiate through a derived type that itself is not marked as abstract).

Check the link: Can an abstract class have a constructor?

- **Interfaces:** Multiple inheritance is not available in java. Instead we have java interfaces. they have abstract functions (no body of functions)
- The **interface** is like a class but not completely. It is like an abstract class.
- By default functions are public and abstract in interface.

- variables are final and static by default in the interface.
- Interfaces specify only what the class is doing, not how it is doing it.
- The problem with MULTIPLE INHERITANCE is that two classes may define different ways of doing the same thing, and the subclass can't choose which one to pick.
- **Key difference between a class and an interface:** a class can maintain state information (especially through the use of instance variables), but an interface cannot.
- Using interface, you can specify a set of methods that can be implemented by one or more classes.
- Although they are similar to abstract classes, interfaces have an additional capability: A class can implement more than one interface. By contrast, a class can only inherit a single superclass (abstract or otherwise).
- Using the keyword interface, you can fully abstract a class' interface from its implementation. That is, using interface, you can specify what a class must do, but not how it does it.
- Interfaces are syntactically similar to classes, but they lack instance variables, and, as a general rule, their methods are declared without any body.
- By providing the interface keyword, Java allows you to fully utilize the "one interface, multiple methods" aspect of polymorphism.
- **NOTE:** Interfaces are designed to support dynamic method resolution at run time.
- Normally, in order for a method to be called from one class to another, both classes need to be present at compile time so the Java compiler can check to ensure that the method signatures are compatible. This requirement by itself makes for a static and nonextensible classing environment. Inevitably in a system like this, functionality gets pushed up higher and higher in the class hierarchy so that the mechanisms will be available to more and more subclasses.
- Interfaces are designed to avoid this problem. They disconnect the definition of a method or set of methods from the inheritance hierarchy.
- Since interfaces are in a different hierarchy from classes, it is possible for classes that are unrelated in terms of the class hierarchy to implement the same interface. This is where the real power of interfaces is realized.

- Beginning with JDK 8, it is possible to add a default implementation to an interface method.
- Thus, it is now possible for interface to specify some behavior. However, default methods constitute what is, in essence, a special-use feature, and the original intent behind the interface still remains.
- Variables can be declared inside of interface declarations.
- **NOTE:** They are implicitly final and static, meaning they cannot be changed by the implementing class. They must also be initialized. All methods and variables are implicitly public.
- **NOTE:** The methods that implement an interface must be declared public. Also, the type signature of the implementing method must match exactly the type signature specified in the interface definition.
- It is both permissible and common for classes that implement interfaces to define additional members of their own.
- **NOTE:** You can declare variables as object references that use an interface rather than a class type.
- This process is similar to using a superclass reference to access a subclass object.
- Any instance of any class that implements the declared interface can be referred to by such a variable.
- When you call a method through one of these references, the correct version will be called based on the actual instance of the interface being referred to. Called at run time by the type of object it refers to.
- The method to be executed is looked up dynamically at run time, allowing classes to be created later than the code which calls methods on them.
- The calling code can dispatch through an interface without having to know anything about the “callee.”
- **CAUTION:** Because dynamic lookup of a method at run time incurs a significant overhead when compared with the normal method invocation in Java, you should be careful not to use interfaces casually in performance-critical code.
- **Nested Interfaces:** An interface can be declared a member of a class or another interface. Such an interface is called a member interface or a nested interface.

- A nested interface can be declared as public, private, or protected.
- This differs from a top-level interface, which must either be declared as public or use the default access level.

```
class A { // This class contains a member interface.
  // this is a nested interface
  public interface NestedIF {
    boolean isNotNegative(int x);
  }
}
// B implements the nested interface.
class B implements A.NestedIF {
  public boolean isNotNegative(int x) {
    return x < 0 ? false: true;
  }
}
class NestedIFDemo {
  public static void main(String args[]) {
    // use a nested interface reference
    A.NestedIF nif = new B();
    if(nif.isNotNegative(10))
      System.out.println("10 is not negative");
    if(nif.isNotNegative(-12))
      System.out.println("this won't be displayed");
  }
}
```

- **Interfaces Can Be Extended:** One interface can inherit another by use of the keyword extends. The syntax is the same as for inheriting classes.

- Any class that implements an interface must implement all methods required by that interface, including any that are inherited from other interfaces.
- **Default Interface Methods (aka extension method) :** A primary motivation for the default method was to provide a means by which interfaces could be expanded without breaking existing code.
- i.e. suppose you add another method without body in an interface. Then you will have to provide the body of that method in all the classes that implement that interface.
- **Example:**

  ```
  default String getString() {
    return "Default String";
  }
  ```

- For example, you might have a class that implements two interfaces. If each of these interfaces provides default methods, then some behavior is inherited from both.

In all cases, a class implementation takes priority over an interface default implementation.

In cases in which a class implements two interfaces that both have the same default method, but the class does not override that method, then an error will result.

In cases in which one interface inherits another, with both defining a common default method, the inheriting interface's version of the method takes precedence.

- **NOTE:** static interface methods are not inherited by either an implementing class or a subinterface. i.e. static interface methods should have a body! They cannot be abstract.
- **NOTE :** when overriding methods, the access modifier should be same or better i.e. if in Parent Class it was protected, then then overridden should be either protected or public.
-

Interfaces vs Abstract class

Type of methods:

Interfaces:

- Traditionally, interfaces contained **only abstract methods**.

- From **Java 8**, interfaces can also contain:
    - **default methods**
    - **static methods**
- From **Java 9**, interfaces can also contain **private methods**.
- So an interface can have:

1. abstract methods
2. default methods
3. static methods
4. private methods (Java 9+)

Abstract Class:

- An **abstract class** can contain:
    - **abstract methods** (methods without body)
    - **non-abstract methods** (methods with implementation)
    - Therefore, abstract classes can have **both implemented and non-implemented methods**.

Final Variables:

- *Interface :* Variables declared in an interface are by default ***`public, static and final.`*** So they behave like constants.
- ***Example****:*

interface A{

int x = 10;

}

public static final int x = 10; //This is automatically treated.

- *Abstract Class:* An abstract class may contain non-final variables. Variables can be modified unless declared `final`.

Example:

```
abstract class A{
   int x = 10;
}
```

Type of variables:

- *Abstract Class:* An abstract class can contain final variables, non-final variables, static variables, non-static variables.
- So abstract classes behave like normal classes with additional abstraction.
- *Interface:* Interfaces can contain only static and final variables.
- No instance variables are allowed.

Implementation:

- *Abstract Class:* Abstract class can provide the implementation of interface.
- ***Example:***

  ```
  interface A{}
  abstract class B implements A{}
  ```

- *Interface:* Interfaces can't provide the implementation of abstract classes. Interfaces only extend other interfaces.

Inheritance vs Abstraction:

- *Interface:* A Java interface can be implemented using keyword "implements"
- *Abstract Class:* abstract class can be extended using keyword "extends".

Multiple implementation:

- *Interface:* An interface can **extend another interface only**.
- *Abstract Class:* An abstract class can:

1. **extend another class**
2. **implement multiple interfaces**

Example:

abstract class A extends B implements C, D

Accessibility of Data Members:

- *Interface:* Members of a Java interface are public by default.
- *Abstract Class:* A Java abstract class can have class members like private, protected, public, default.

Feature	Interface	Abstract Class
Methods	abstract, default, static (Java 8+), private (Java 9+)	abstract + concrete
Variables	public static final only	any type
Object creation	cannot create	cannot create
Inheritance keyword	implements	extends
Multiple inheritance	yes	only single class inheritance

6. Generics, Custom ArrayList, Lambda Expressions, Exception Handling, Object cloning.

- **1. Generics (Compile-Time Type Safety):** Generics allow you to parameterize types, meaning classes, interfaces, and methods can operate on objects of various types while providing compile-time type safety.
- **Mechanism:** You use angle brackets <T> to specify a type parameter. This eliminates the need for explicit casting and catches ClassCastException errors at compile-time rather than runtime.
- **Bounded Types:** You can restrict the types that can be passed to a generic type parameter using the extends keyword (e.g., <T extends Number>).

- **2. Custom ArrayList (Dynamic Data Structures):** A standard ArrayList is a dynamic array that grows automatically. Building a custom version involves wrapping a standard array inside a class and managing its capacity manually.
- **Implementation:** It requires an underlying Object[] array. When the array reaches its capacity limit, a new, larger array is created, and the old elements are copied over (often using Arrays.copyOf()).
- **Integration with Generics:** A custom ArrayList must be made generic (e.g., CustomArrayList<T>) so it can store any object type securely.
- **3. Lambda Expressions (Functional Programming):** Introduced in Java 8, lambda expressions provide a clear and concise way to represent one-method interfaces (Functional Interfaces) using an expression.
- **Syntax:** (parameters) -> { body }.
- **Usage:** They are predominantly used to pass behaviors as parameters to methods, such as iterating over collections, filtering data, or handling events in GUIs.
- **4. Exception Handling (Robustness):** Exception handling is a powerful mechanism that handles runtime errors, maintaining the normal flow of the application.
- **Hierarchy:** At the top is Throwable, branched into Error (severe problems like OutOfMemoryError) and Exception (recoverable conditions). Exceptions are further divided into Checked (compile-time) and Unchecked (RuntimeException, logical errors).
- **Keywords:** try (code that might throw), catch (handling the throw), finally (guaranteed execution, often for resource cleanup), throw (explicitly throwing an exception), and throws (declaring that a method might throw).
- **5. Object Cloning (Duplicating State):** Object cloning refers to creating an exact copy of an object. In Java, this is done using the clone() method of the Object class.
- **Marker Interface:** A class must implement the Cloneable interface; otherwise, the clone() method throws a CloneNotSupportedException.
- **Shallow vs. Deep Copy:**
- *Shallow Copy:* The default behavior of clone(). It copies primitive values directly but only copies the *references* of nested objects. The original and cloned object will share the same nested objects.

- *Deep Copy:* Requires manually overriding the clone() method to also explicitly clone all nested objects, ensuring complete independence.

Remaining topics*: *Collection framework, Enum, Vector classes are simple. Just check the code.